Shopify AI Customer Support Automation

AI Researcher / AI Innovation Engineer Assignment

Executive Summary

Customer Support in E-commerce platforms is generally sluggish, involve repetitive queries and regarding, shipping, returns, defects, and more. Having to process these requests individually costs a lot of time, effort and adds operational cost to the company.

This project involves a AI powered customer support system for shopify, the solution combines LLMs, business rule evaluation, RAG and RESTful API to automate ticket classification, generation of response while also ensuring that all high priority/risk requests are escalated to human agents

This system was created using Python, Gemini API and FastAPI.

1. Recommended Architecture

The workflow starts when a customer enters their ticket in the FastAPI endpoint, which is the forwarded to a classification system that is powered by Gemini. This classification system understands the ticket based on intent, sentiment, priority level, escalation requirement. This output is then passed on to the business rules layer which contains policies related to the company.

E.g. If the request is related to a damaged product or something that exceeds the thresholds, they will automatically be marked for a human review. This will ensure that customer issue is not handled solely by AI.

If the ticket passes to a business rule layer, the system retrieves information that contains the policies of company. In this prototype the policies (knowledge base) is in a local text files having refund, shipping and replacement policies. This retrieval step acts as a RAG (Retrieval Augmented Generation) layer and provides factual details to the LLM.

Following that, the policies and the ticket is passed to a Gemini response generation model. The generated response is returned to the FastAPI service as a well structure JSON object containing the classification, escalation and customer response.

This architecture was chosen because separates business logic, classification, retrieval and response generation into separate components. This helps maintain structure, testing and scalability, if required for the future.

2. Why Specific Tools and Models Were Selected

Multiple platforms were tested during the research including, OpenAI, Gemini and Claude.

Gemini was selected as the primary model since it had a good balance between performance, developmental cost and ease of accessibility. The API was extremely straightforward to use and easy to integrate to python application. Since the task at hand was not too complex, Gemini was fairly sufficient for basic classification, sentiment analysis, response generation and interpreting the policy as required.

I selected FastAPI since the framework it runs on is super lightweight, perfant and also automatically generated API documentation. This makes it easy to demonstrate the prototype for output during development.

Python was chosen since it is widely used in AI space and is highly compatible with modern framework and API. Also, the language enable fast prototyping while also keeping the task complexity low.

I choose RAG approach to avoid/reduce hallucinations since relying on a AI model alone can be risky. By checking that the model first retrieves the policies and then generate its answers, I effectively lowered the amount of hallucination that the model can make.

3. Estimated Infrastructure Cost

The prototype was designed to be run at minimal cost.

During the development, the system runs entirely on local server and uses the free tier of Gemini API. Because of this the development cost is zero barring the internet connections and hardware; which is already possessed by the dev.

For a production scale, the primary cost will be related to cloud services, AI interference and Domain. A small scale FastAPI application can be hosted on a low cost server; however, the usage for Gemini API would scale according to demand in ticket.

For a small scaled Shopify store handling around some hundreds of requests, I have broken down the stats as follows:

According to Google's Gemini API pricing documentation, Gemini 2.5 Flash is priced at approximately **$0.30 per million input tokens** and **$2.50 per million output tokens (**source**)**. Google's pricing model is based on tokens therefore; the costs might increase as per the text processed instead of the number of requests.

For example, let's assume:

- 300 tickets per day
- 1,000 input tokens per ticket
- 500 output tokens per ticket
- Two AI calls per workflow

Approx monthly consumption:

- 18 million input tokens
- 9 million output tokens

Therefore,

- Input cost: 18 × $0.30 = $5.40
- Output cost: 9 × $2.50 = $22.50

Estimated Cost (AI only): $28 per month

Cost of hosting application is likely to remain low since the FastAPI service can be deployed on a small cloud or a virtual machine for approx. $5 to $15 per month. In total, the Shopify business would fall in the range of around $30-$50 per month (as per the model section and hosting).

(Also, to prevent priority fails for Gemini requests, developer might consider upgrading the plan to higher priority levels. However, to maintain low-cost estimates, only the standard version of the model is considered.)

4. Risks and Limitations

The dependence of the system on external AI service is its primary limitation. If the API service experience any outage, quota restrictions, or rate limitation, the overall working of the application will become unavailable. To counter this, we can redirect all services to a human agent in this case.

Another limitation will be the possibility of hallucination. Even though a RAG layer has been implemented, the model still can provide incorrect outputs if the policies are not correctly defined.

For the current, short-term deployment, we are using a static knowledge base file (store_policies.txt). This might be sufficient for a prototype; however, for a larger production scale environment, this should be shifted towards a vector-based database.

Finally, an automated, real-time notification system (Webhook) for Shopify is not included. I believe for a demonstration of prototype this could be considered acceptable

5. Production Scalability Strategy

For productions, we could ensure that the support be delivered using webhooks instead of processing requests on at a type. We could make use of parallel processing where multiple tickets are simultaneously put in queue and solved accordingly thereby, reducing traffic in the app.

Changing the static policy file to a vector database such as Pinecone or Chroma DB would help in upgrading and enhancing RAG layer. These databases would have thousands of documents and queries to access from.

Human intervention could also be integrated in the loop to ensure a QA process before sending the response of the ticket directly to the customer. This could provide as additional layer of value however, also increase traffic in case the efficiency of the person involved is not up to the mark.